

5.1. 相机校正和图像校正

学习目标

- 知道相机校正API及其使用方法
 - 了解图像去畸变的方法
-

1.相机标定

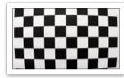
根据张正友校正算法，利用棋盘格数据校正对车载相机进行校正，计算其内参矩阵，外参矩阵和畸变系数。

标定的流程是：

- 准备棋盘格数据，即用于标定的图片
- 对每一张图片提取角点信息
- 在棋盘上绘制提取到的角点（非必须，只是为了显示结果）
- 利用提取的角点对相机进行标定
- 获取相机的参数信息

1.1. 标定的图片

标定的图片需要使用棋盘格数据在不同位置、不同角度、不同姿态下拍摄的图片，最少需要3张，当然多多益善，通常是10-20张。该项目中我们使用了20张图片，如下图所示：



calibration1.jpg



calibration2.jpg



calibration3.jpg



calibration4.jpg



calibration5.jpg



calibration6.jpg



calibration7.jpg



calibration8.jpg



calibration9.jpg



calibration10.jpg



calibration11.jpg



calibration12.jpg



calibration13.jpg



calibration14.jpg



calibration15.jpg



calibration16.jpg



calibration17.jpg



calibration18.jpg



calibration19.jpg



calibration20.jpg

把这些图片存放在项目路径中的camera_cal文件夹中。

1.2. 相机校正

下面我们对相机进行校正，OPenCV中提供了对相机进行校正的代码，在本项目中直接使用opencv中的API进行相机的校正，如下所示：

```

# 1. 参数设定:定义棋盘横向和纵向的角点个数并指定校正图像的位置
nx = 9
ny = 6
file_paths = glob.glob("./camera_cal/calibration*.jpg")
# 2. 计算相机的内外参数及畸变系数
def cal_calibrate_params(file_paths):
    object_points = [] # 三维空间中的点: 3D
    image_points = [] # 图像空间中的点: 2d
    # 2.1 生成真实的交点坐标: 类似(0,0,0), (1,0,0), (2,0,0) ....., (6,5,0)的三维点
    objp = np.zeros((nx * ny, 3), np.float32)
    objp[:, :2] = np.mgrid[0:nx, 0:ny].T.reshape(-1, 2)
    # 2.2 检测每幅图像角点坐标
    for file_path in file_paths:
        img = cv2.imread(file_path)
        # 将图像转换为灰度图
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        # 自动检测棋盘格内4个棋盘格的角点 (2白2黑的交点)
        rect, corners = cv2.findChessboardCorners(gray, (nx, ny), None)
        # 若检测到角点, 则将其存储到object_points和image_points
        if rect == True:
            object_points.append(objp)
            image_points.append(corners)
    # 2.3 获取相机参数
    ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(object_points, image_po
    return ret, mtx, dist, rvecs, tvecs

```

在这里有几个API给大家介绍下:

1. 寻找棋盘图中的棋盘角点

```
rect, corners = cv2.findChessboardCorners(image, pattern_size, flags)
```

参数:

- Image: 输入的棋盘图, 必须是8位的灰度或者彩色图像
- Pattern_size: 棋盘图中每行每列的角点个数 (内角点)。
- flags: 用来定义额外的滤波步骤以有助于寻找棋盘角点。所有的变量都可以单独或者以逻辑或的方式组合使用。取值主要有:

CV_CALIB_CB_ADAPTIVE_THRESH: 使用自适应阈值 (通过平均图像亮度计算得到) 将图像转换为黑白图, 而不是一个固定的阈值。

CV_CALIB_CB_NORMALIZE_IMAGE: 在利用固定阈值或者自适应的阈值进行二值化之前, 先使用cvNormalizeHist来均衡化图像亮度。

CV_CALIB_CB_FILTER_QUADS: 使用其他的准则 (如轮廓面积, 周长, 方形形状) 来去除在轮廓检测阶段检测到的错误方块。

返回:

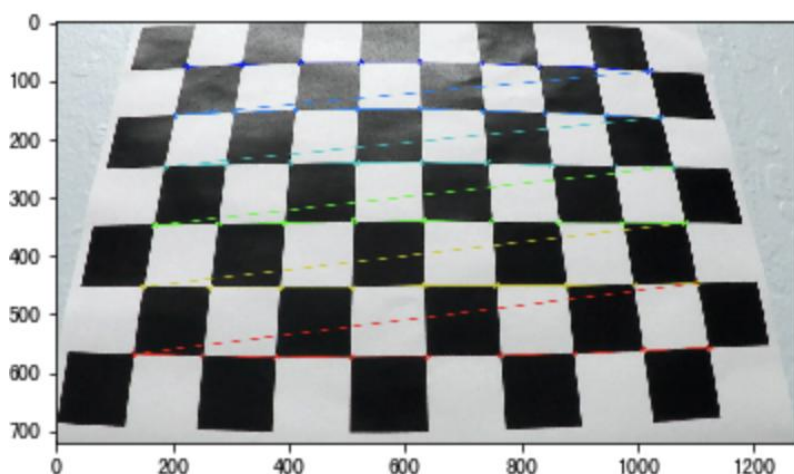
- Corners: 检测到的角点
- rect: 输出是否找到角点, 找到角点返回1, 否则返回0
- 检测完角点之后我们可以将测到的角点绘制在图像上, 使用的API是:

```
cv2.drawChessboardCorners(img, pattern_size, corners, rect)
```

参数：

- `Img`: 预绘制检测角点的图像
- `pattern_size`: 预绘制的角点的形状
- `corners`: 角点矩阵
- `rect`: 表示是否所有的棋盘角点被找到，可以设置为 `findChessboardCorners` 的返回值

注意：如果发现了所有的角点，那么角点将用不同颜色绘制（每行使用单独的颜色绘制），并且把角点以一定顺序用线连接起来，如下图所示：



- 利用定标的结果计算内外参数

```
ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(object_points, image_points
```

参数：

- `Object_points`: 世界坐标系中的点，在使用棋盘的场合，我们令 `z` 的坐标值为 0，而 `x, y` 坐标用里面来度量，选用英寸单位，那么所有参数计算的结果也是用英寸表示。最简单的方式是我们定义棋盘的每一个方块为一个单位。
- `image_points`: 在图像中寻找到的角点的坐标，包含 `object_points` 所提供的所有点
- `image_size`: 图像的大小，以像素为衡量单位

返回：

- `ret`: 返回值
- `mtx`: 相机的内参矩阵，大小为 3×3 的矩阵
- `dist`: 畸变系数，为 5×1 大小的矢量
- `rvecs`: 旋转变量
- `tvecs`: 平移变量

1.3 图像去畸变

上一步中我们已经得到相机的内参及畸变系数，我们利用其进行图像的去畸变，最直接的方法就是调用 `opencv` 中的函数得到去畸变的图像：

```
def img_undistort(img, mtx, dist):
    dst = cv2.undistort(img, mtx, dist, None, mtx)
    return dst
```

我们看下求畸变的API:

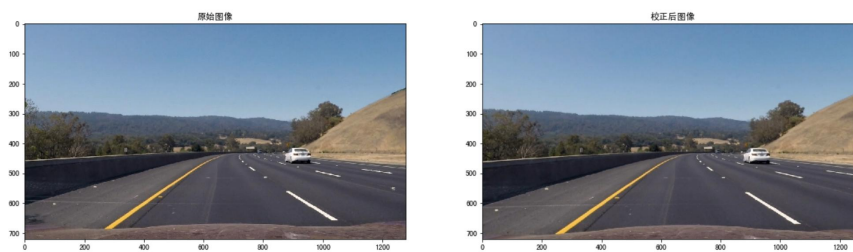
```
dst = cv2.undistort(img, mtx, dist, None, mtx)
```

参数:

- img: 要进行校正的图像
- mtx: 相机的内参
- dist: 相机的畸变系数

返回:

- dst: 图像校正后的结果



总结:

1. 标定的图片

不同角度和方向拍摄的棋盘格图片数据

2. 相机校正

检测棋盘格数据的角点:

`cv2.findChessboardCorners ()`

计算相机的内参数和外参数: `cv2.calibrateCamera ()`

3. 图像去畸变

`cv2.undistort()`

5.2. 车道线提取

学习目标

- 了解HLS颜色空间
- 了解车道线提取的方法

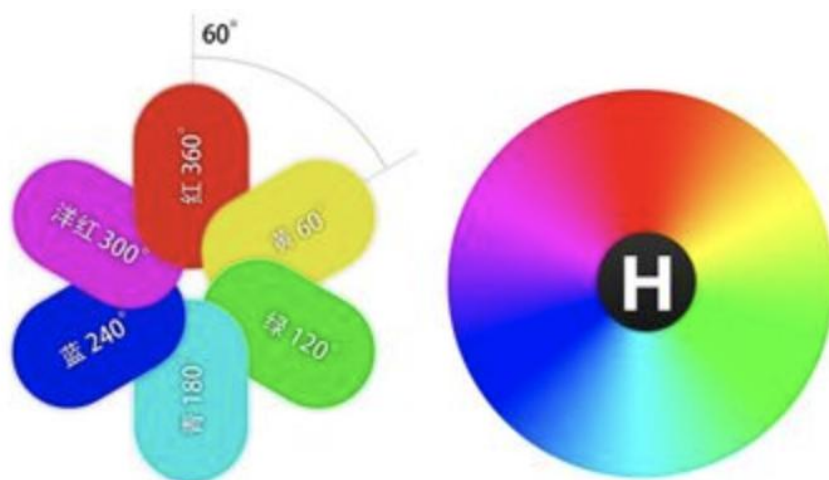
我们基于图像的梯度和颜色特征，定位车道线的位置。

在这里选用Sobel边缘提取算法，Sobel相比于Canny的优秀之处在于，它可以选择横向或纵向的边缘进行提取。从车道的拍摄图像可以看出，我们关心的正是车道线在横向上的边缘突变。OpenCV提供的cv2.Sobel()函数，将进行边缘提取后的图像做二进制图的转化，即提取到边缘的像素点显示为白色（值为1），未提取到边缘的像素点显示为黑色（值为0）。由于只使用边缘检测，在有树木阴影覆盖的区域时，虽然能提取出车道线的大致轮廓，但会同时引入的噪声，给后续处理带来麻烦。所以在这里我们引入颜色阈值来解决这个问题。

1.颜色空间

在车道线检测中，我们使用的是HSL颜色空间，其中H表示色相，即颜色，S表示饱和度，即颜色的纯度，L表示颜色的明亮程度。

HSL的H(hue)分量，代表的是人眼所能感知的颜色范围，这些颜色分布在一个平面的色相环上，取值范围是0°到360°的圆心角，每个角度可以代表一种颜色。色相值的意义在于，我们可以在不改变光感的情况下，通过旋转色相环来改变颜色。在实际应用中，我们需要记住色相环上的六大主色，用作基本参照：360°/0°红、60°黄、120°绿、180°青、240°蓝、300°洋红，它们在色相环上按照60°圆心角的间隔排列：



HSL的S(saturation)分量，指的是色彩的饱和度，描述了相同色相、明度下色彩纯度的变化。数值越大，颜色中的灰色越少，颜色越鲜艳，呈现一种从灰度到纯色的变化。因为车道线是黄色或白色，所以利用s通道阈值来检测车道线。



HSL的L(lightness)分量，指的是色彩的明度，作用是控制色彩的明暗变化。数值越小，色彩越暗，越接近于黑色；数值越大，色彩越亮，越接近于白色。



2.车道线提取

车道线提取的代码如下所示：

```
# 车道线提取代码
def pipeline(img, s_thresh=(170, 255), sx_thresh=(40, 200)):

    img = np.copy(img)
    #1.将图像转换为HLS色彩空间，并分离各个通道
    hls = cv2.cvtColor(img, cv2.COLOR_RGB2HLS).astype(np.float)
    h_channel = hls[:, :, 0]
    l_channel = hls[:, :, 1]
    s_channel = hls[:, :, 2]
    #2.利用sobel计算x方向的梯度
    sobelx = cv2.Sobel(l_channel, cv2.CV_64F, 1, 0)
    abs_sobelx = np.absolute(sobelx)
    # 将导数转换为8bit整数
    scaled_sobel = np.uint8(255 * abs_sobelx / np.max(abs_sobelx))
    sxbinary = np.zeros_like(scaled_sobel)

    sxbinary[(scaled_sobel >= sx_thresh[0]) & (scaled_sobel <= sx_thresh[1])] = 1

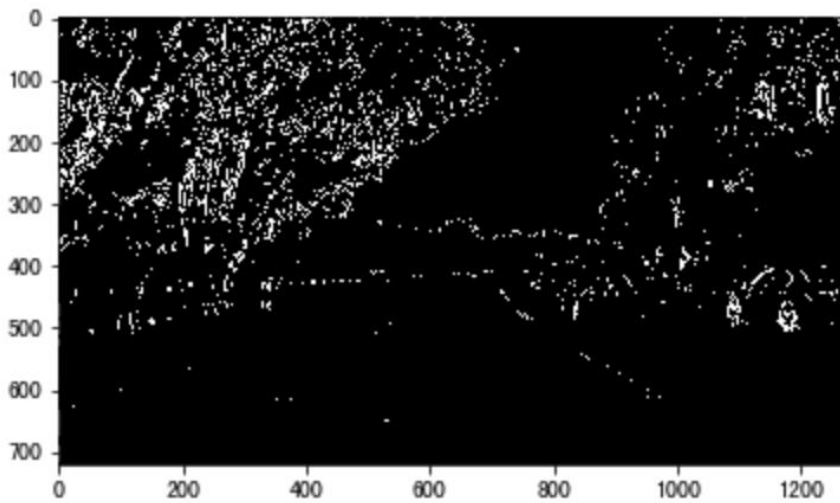
    # 3.对s通道进行阈值处理
    s_binary = np.zeros_like(s_channel)
    s_binary[(s_channel >= s_thresh[0]) & (s_channel <= s_thresh[1])] = 1
    # 4. 将边缘检测的结果和颜色空间阈值的结果合并，并结合l通道的取值，确定车道提取的二值图
    color_binary = np.zeros_like(sxbinary)
    color_binary[((sxbinary == 1) | (s_binary == 1)) & (l_channel > 100)] = 1
    return color_binary
```

我们来看下整个流程：

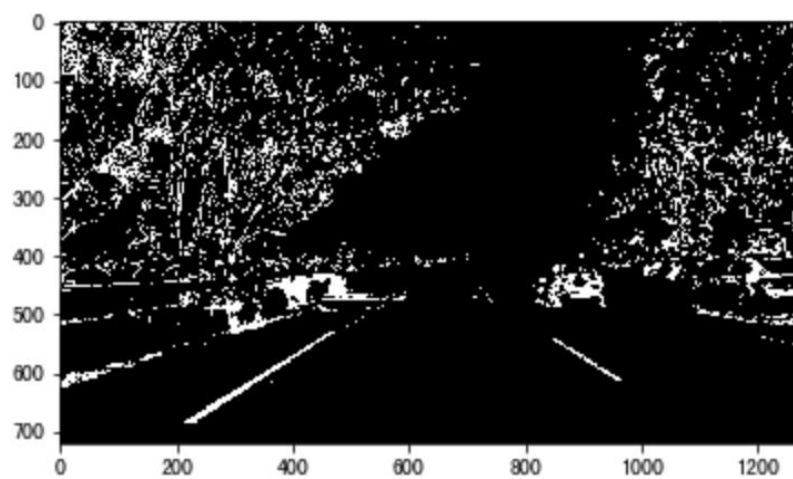
首先我们是把图像转换为HLS颜色空间，然后利用边缘检测和阈值的方法检测车道线，我们以下图为例，来看下检测结果：



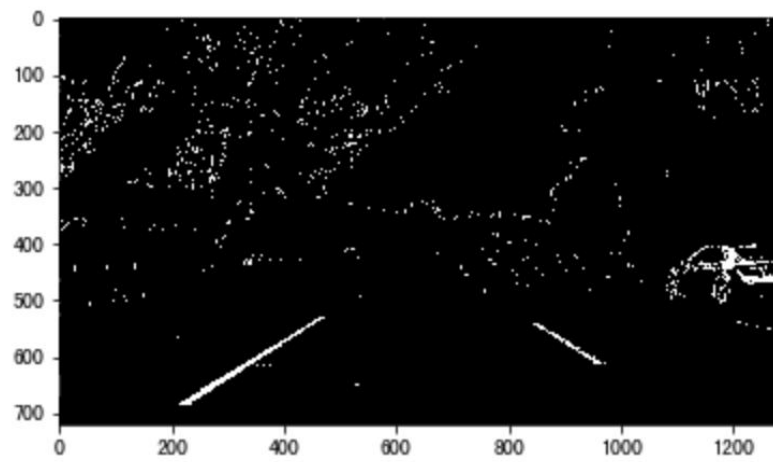
- 利用sobel边缘检测的结果



- 利用S通道的阈值检测结果



- 将边缘检测结果与颜色检测结果合并，并利用L通道抑制非白色的信息：



总结：

1. 颜色空间：

HLS：色相，饱和度，明亮程度

2. 车道线提取

颜色空间转换-》边缘检测-》颜色阈值-》合并得到检测结果。

5.3. 透视变换

学习目标

- 知道透视变换在车道线检测中的应用

为了方便后续的直方图滑窗对车道线进行准确的定位，我们在这里利用透视变换将图像转换成俯视图，也可将俯视图恢复成原有的图像，代码如下：

计算透视变换所需的参数矩阵：

```
def cal_perspective_params(img, points):
    offset_x = 330
    offset_y = 0
    img_size = (img.shape[1], img.shape[0])
    src = np.float32(points)
    # 俯视图中四点的位置
    dst = np.float32([[offset_x, offset_y], [img_size[0] - offset_x, offset_y],
                     [offset_x, img_size[1] - offset_y],
                     [img_size[0] - offset_x, img_size[1] - offset_y]])
    # 从原始图像转换为俯视图的透视变换的参数矩阵
    M = cv2.getPerspectiveTransform(src, dst)
    # 从俯视图转换为原始图像的透视变换参数矩阵
    M_inverse = cv2.getPerspectiveTransform(dst, src)
    return M, M_inverse
```

透视变换：

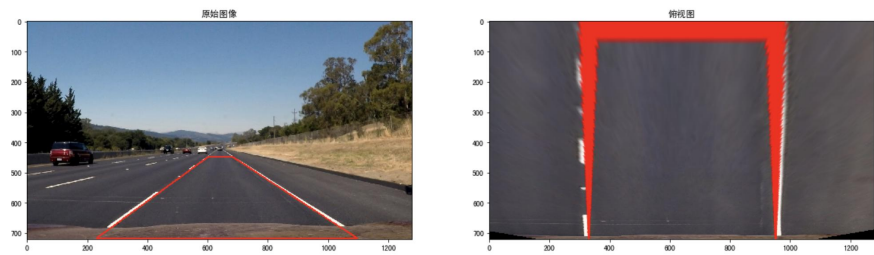
```
def img_perspect_transform(img, M):
    img_size = (img.shape[1], img.shape[0])
    return cv2.warpPerspective(img, M, img_size)
```

下面我们调用上述两个方法看下透视变换的结果：

在原始图像中我们绘制道路检测的结果，然后通过透视变换转换为俯视图。

```
img = cv2.imread("./test/straight_lines2.jpg")
img = cv2.line(img, (601, 448), (683, 448), (0, 0, 255), 3)
img = cv2.line(img, (683, 448), (1097, 717), (0, 0, 255), 3)
img = cv2.line(img, (1097, 717), (230, 717), (0, 0, 255), 3)
img = cv2.line(img, (230, 717), (601, 448), (0, 0, 255), 3)
points = [[601, 448], [683, 448], [230, 717], [1097, 717]]
M, M_inverse = cal_perspective_params(img, points)
transform_img = img_perspect_transform(img, M)
plt.figure(figsize=(20, 8))
plt.subplot(1, 2, 1)
plt.title('原始图像')
plt.imshow(img[:, :, :-1])
plt.subplot(1, 2, 2)
plt.title('俯视图')
plt.imshow(transform_img[:, :, :-1])
plt.show()
```

2.1. numba



总结：

透视变换将检测结果转换为俯视图。

5.4. 车道线定位及拟合

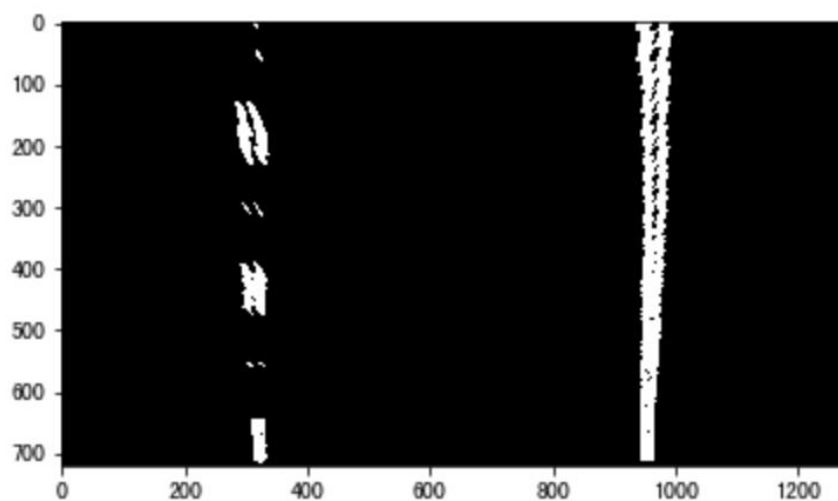
学习目标

- 了解直方图确定车道线位置的思想

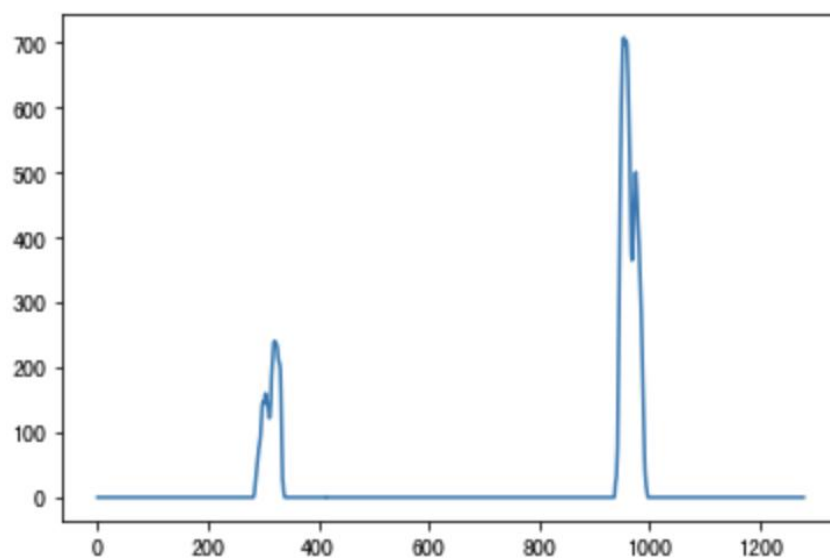
我们根据前面检测出的车道线信息，利用直方图和滑动窗口的方法，精确定位车道线，并进行拟合。

1. 定位思想

下图是我们检测到的车道线结果：



沿x轴方向统计每一列中白色像素点的个数，横坐标是图像的列数，纵坐标表示每列中白色点的数量，那么这幅图就是“直方图”，如下图所示：

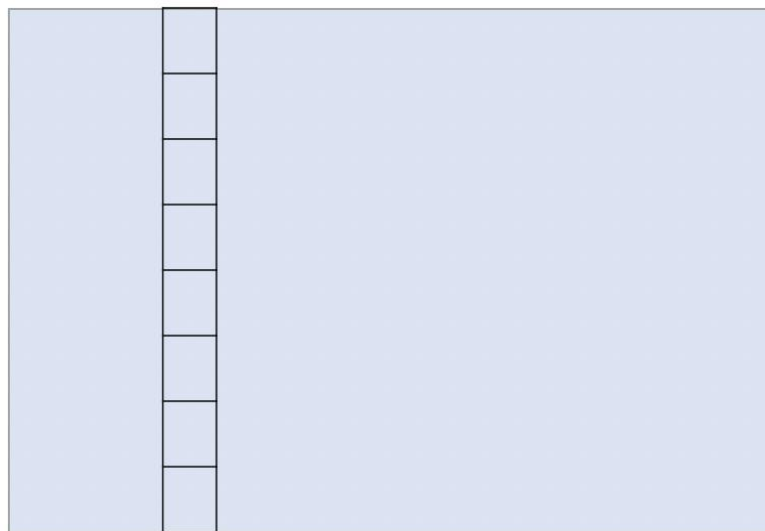


对比上述两图，可以发现直方图左半边最大值对应的列数，即为左车道线所在的位置，直方图右半边最大值对应的列数，是右车道线所在的位置。

确定左右车道线的大致位置后，使用“滑动窗口”的方法，在图中对左右车道线的点进行搜索。

滑动窗口的搜索过程：

- 设置搜索窗口大小（width和height）：一般情况下width为手工设定，height为图片大小除以设置搜索窗口数目计算得到。
- 以搜寻起始点作为当前搜索的基点，并以当前基点为中心，做一个网格化搜索。



- 对每个搜索窗口分别做水平和垂直方向直方图统计，统计在搜索框区域内非零像素个数，并过滤掉非零像素数目小于50的框。
- 计算非零像素坐标的均值作为当前搜索框的中心，并对这些中心点做一个二阶的多项式拟合，得到当前搜寻对应的车道线曲线参数。

2.实现

使用直方图和滑动窗口检测车道线的代码如下：

```

def cal_line_param(binary_warped):
    # 1. 确定左右车道线的位置
    # 统计直方图
    histogram = np.sum(binary_warped[:, :], axis=0)
    # 在统计结果中找到左右最大的点的位置, 作为左右车道检测的开始点
    # 将统计结果一分为二, 划分为左右两个部分, 分别定位峰值位置, 即为两条车道的搜索位置
    midpoint = np.int(histogram.shape[0] / 2)
    leftx_base = np.argmax(histogram[:midpoint])
    rightx_base = np.argmax(histogram[midpoint:]) + midpoint
    # 2. 滑动窗口检测车道线
    # 设置滑动窗口的数量, 计算每一个窗口的高度
    nwindows = 9
    window_height = np.int(binary_warped.shape[0] / nwindows)
    # 获取图像中不为0的点
    nonzero = binary_warped.nonzero()
    nonzeroy = np.array(nonzero[0])
    nonzerox = np.array(nonzero[1])
    # 车道检测的当前位置
    leftx_current = leftx_base
    rightx_current = rightx_base
    # 设置x的检测范围, 滑动窗口的宽度的一半, 手动指定
    margin = 100
    # 设置最小像素点, 阈值用于统计滑动窗口区域内的非零像素个数, 小于50的窗口不对x的中心值进行
    minpix = 50
    # 用来记录搜索窗口中非零点在nonzeroy和nonzerox中的索引
    left_lane_inds = []
    right_lane_inds = []

    # 遍历该副图像中的每一个窗口
    for window in range(nwindows):
        # 设置窗口的y的检测范围, 因为图像是(行列), shape[0]表示y方向的结果, 上面是0
        win_y_low = binary_warped.shape[0] - (window + 1) * window_height
        win_y_high = binary_warped.shape[0] - window * window_height
        # 左车道x的范围
        win_xleft_low = leftx_current - margin
        win_xleft_high = leftx_current + margin
        # 右车道x的范围
        win_xright_low = rightx_current - margin
        win_xright_high = rightx_current + margin

        # 确定非零点的位置x,y是否在搜索窗口中, 将在搜索窗口内的x,y的索引存入left_lane_inds和right_lane_inds
        good_left_inds = ((nonzeroy >= win_y_low) & (nonzeroy < win_y_high) &
                          (nonzerox >= win_xleft_low) & (nonzerox < win_xleft_high))
        good_right_inds = ((nonzeroy >= win_y_low) & (nonzeroy < win_y_high) &
                           (nonzerox >= win_xright_low) & (nonzerox < win_xright_high))
        left_lane_inds.append(good_left_inds)
        right_lane_inds.append(good_right_inds)

        # 如果获取的点的个数大于最小个数, 则利用其更新滑动窗口在x轴的位置
        if len(good_left_inds) > minpix:
            leftx_current = np.int(np.mean(nonzerox[good_left_inds]))
        if len(good_right_inds) > minpix:
            rightx_current = np.int(np.mean(nonzerox[good_right_inds]))

    # 将检测出的左右车道点转换为array
    left_lane_inds = np.concatenate(left_lane_inds)
    right_lane_inds = np.concatenate(right_lane_inds)

    # 获取检测出的左右车道点在图像中的位置
    leftx = nonzerox[left_lane_inds]
    lefty = nonzeroy[left_lane_inds]
    rightx = nonzerox[right_lane_inds]
    righty = nonzeroy[right_lane_inds]

    # 3. 用曲线拟合检测出的点, 二次多项式拟合, 返回的结果是系数

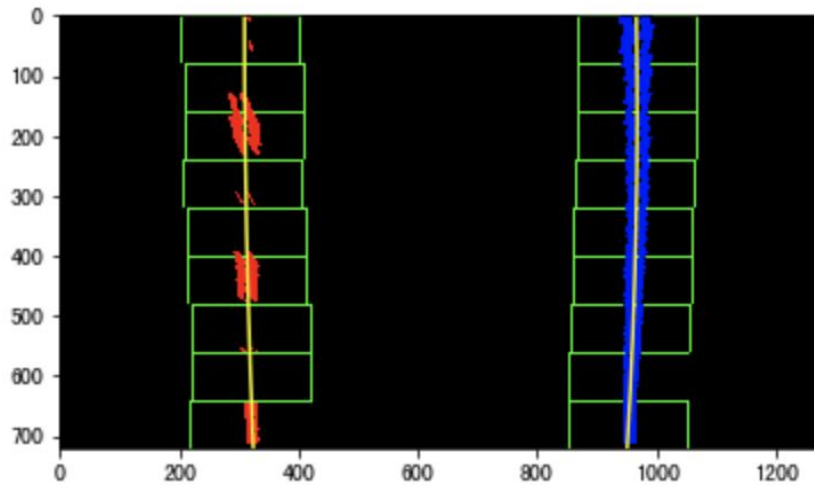
```

```

left_fit = np.polyfit(lefty, leftx, 2)
right_fit = np.polyfit(righty, rightx, 2)
return left_fit, right_fit

```

车道线的检测的拟合结果如下图所示：



其中绿色的方框是滑动窗口的结果，中间的黄线是车道线拟合的结果。

下面我们将车道区域绘制处理，即在检测出的车道线中间绘制多边形，代码如下：

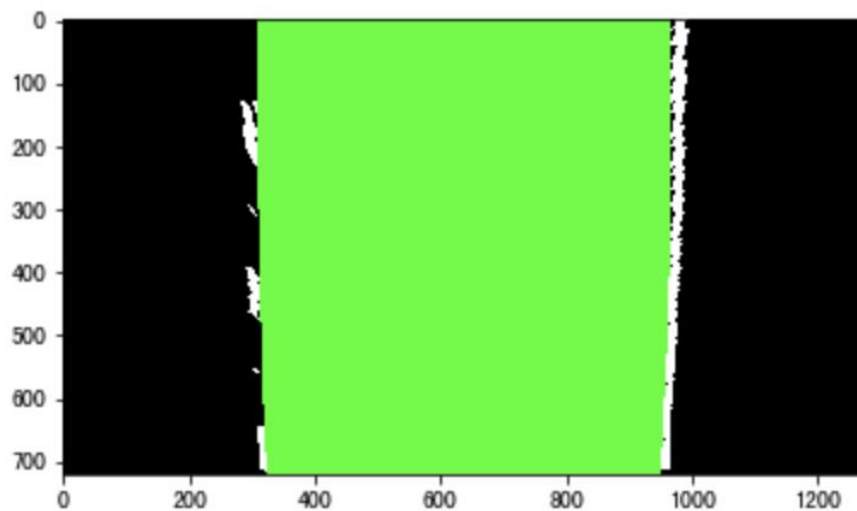
```

def fill_lane_poly(img, left_fit, right_fit):
    # 获取图像的行数
    y_max = img.shape[0]
    # 设置输出图像的大小，并将白色位置设为255
    out_img = np.dstack((img, img, img)) * 255
    # 在拟合曲线中获取左右车道线的像素位置

    left_points = [[left_fit[0] * y ** 2 + left_fit[1] * y + left_fit[2], y]
    right_points = [[right_fit[0] * y ** 2 + right_fit[1] * y + right_fit[2], y]
    # 将左右车道的像素点进行合并
    line_points = np.vstack((left_points, right_points))
    # 根据左右车道线的像素位置绘制多边形
    cv2.fillPoly(out_img, np.int_([line_points]), (0, 255, 0))
    return out_img

```

效果如下图所示：



其中两个方法给大家介绍下：

np.vstack:按垂直方向（行顺序）堆叠数组构成一个新的数组

np.dstack:按水平方向（列顺序）堆叠数组构成一个新的数组

示例如下：

```
In [81]: a = np.array([1, 2, 3])
         b = np.array([4, 5, 6])
```

```
In [83]: np.dstack((a,b))
```

```
Out[83]: array([[1, 4],
                [2, 5],
                [3, 6]])
```

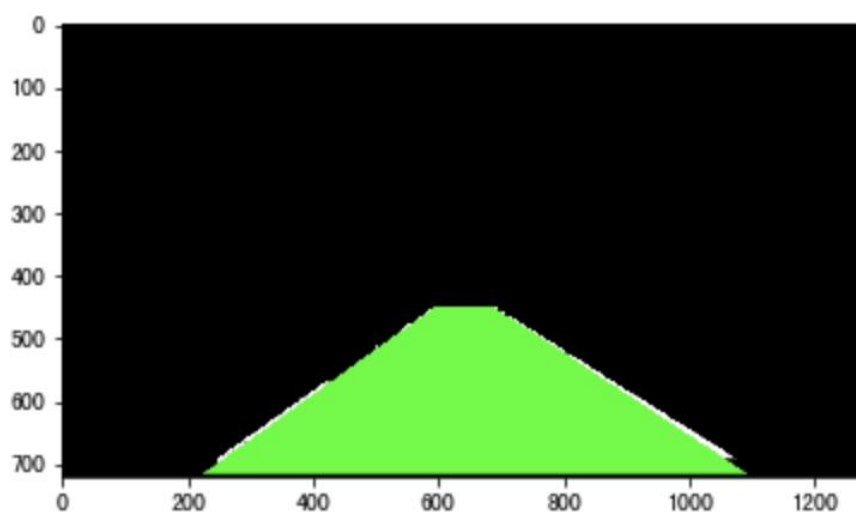
```
In [84]: np.vstack((a,b))
```

```
Out[84]: array([[1, 2, 3],
                [4, 5, 6]])
```

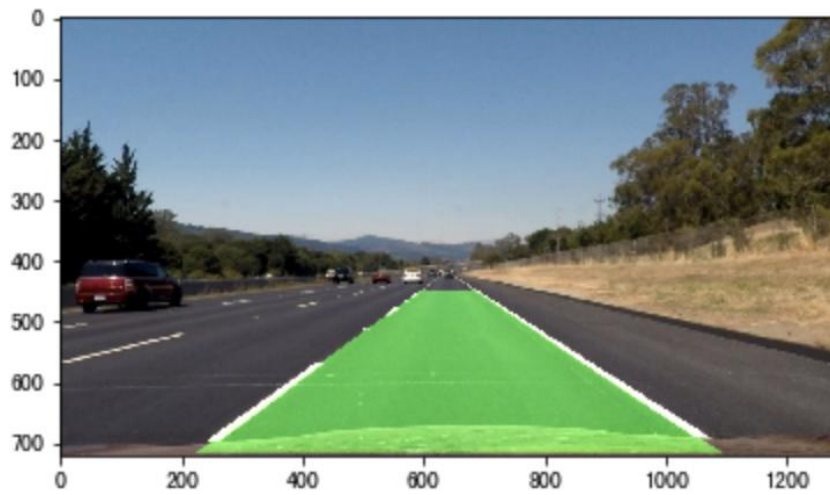
将检测出的车道逆投影到原始图像，直接调用透视变换的方法即可：

```
transform_img_inverse = img_perspective_transform(result, M_inverse)
```

效果如下图所示：



最后将其叠加在原图像上，则有：



总结：

知道车道线精确定位的思想：

首先利用直方图的方法确定左右车道线的位置，然后利用滑动窗口的方法，搜索车道线位置，并进行拟合，然后绘制车道区域，并进行反投影，得到原始图像中的车道区域。

5.5. 车道曲率和中心点偏离距离计算

学习目标

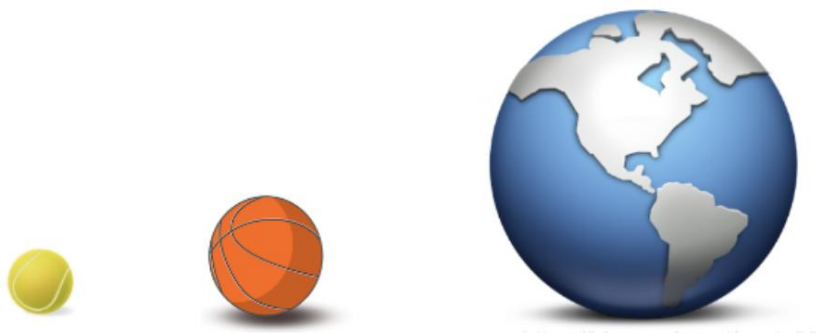
- 知道车道曲率计算的方法
- 知道计算中心点偏离距离的计算

1.曲率的介绍

曲线的曲率就是针对曲线上某个点的切线方向角对弧长的转动率，通过微分来定义，表明曲线偏离直线的程度。数学上表明曲线在某一点的弯曲程度的数值。曲率越大，表示曲线的弯曲程度越大。曲率的倒数就是曲率半径。

1.1.圆的曲率

下面有三个球体，网球、篮球、地球，半径越小的越容易看出是圆的，所以随着半径的增加，圆的程度就越来越弱了。



定义球体或者圆的“圆”的程度，就是 曲率，计算方法为：

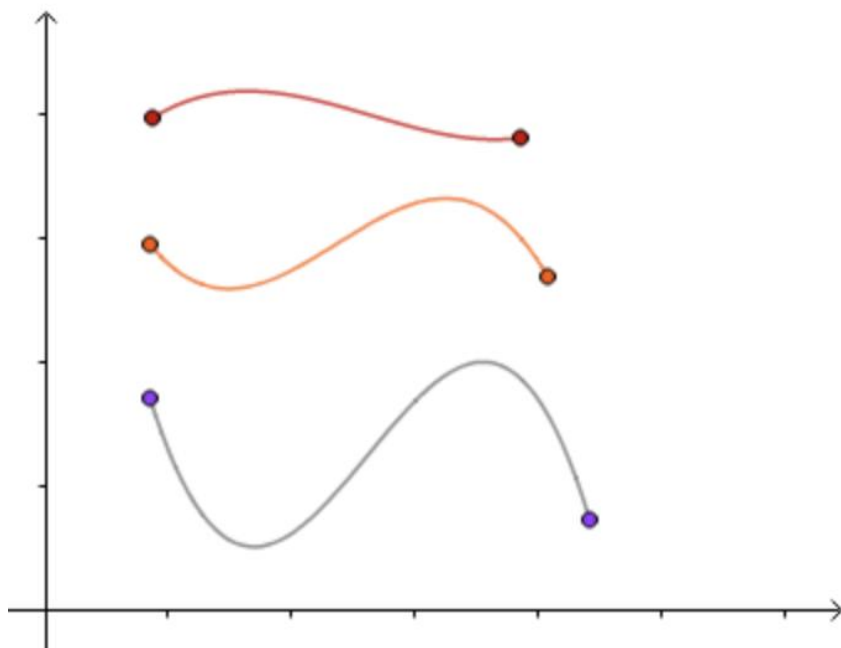
$$K = \frac{1}{r}$$

其中 r 为球体或者圆的半径，这样半径越小的圆曲率越大，直线可以看作半径为无穷大的圆，其曲率为：

$$K = \lim_{r \rightarrow \infty} \frac{1}{r} = 0$$

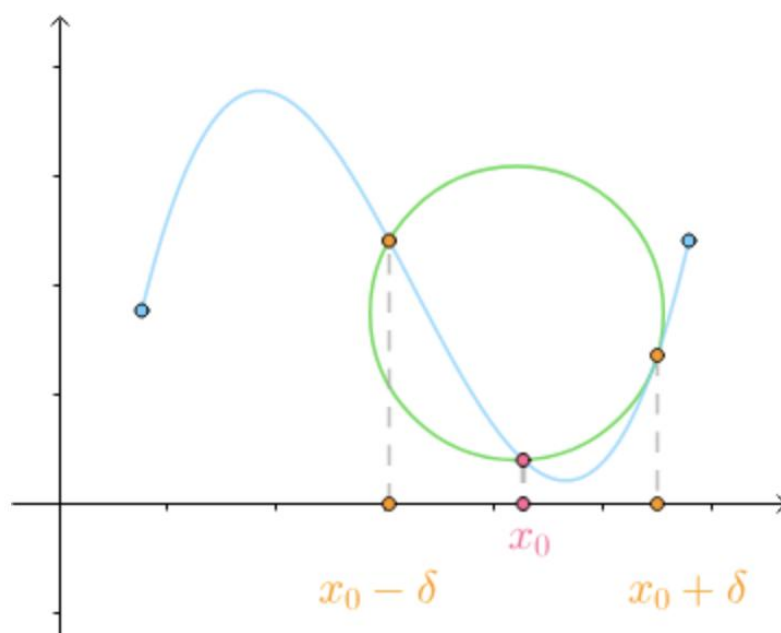
1.2.曲线的曲率

不同的曲线有不同的弯曲程度：

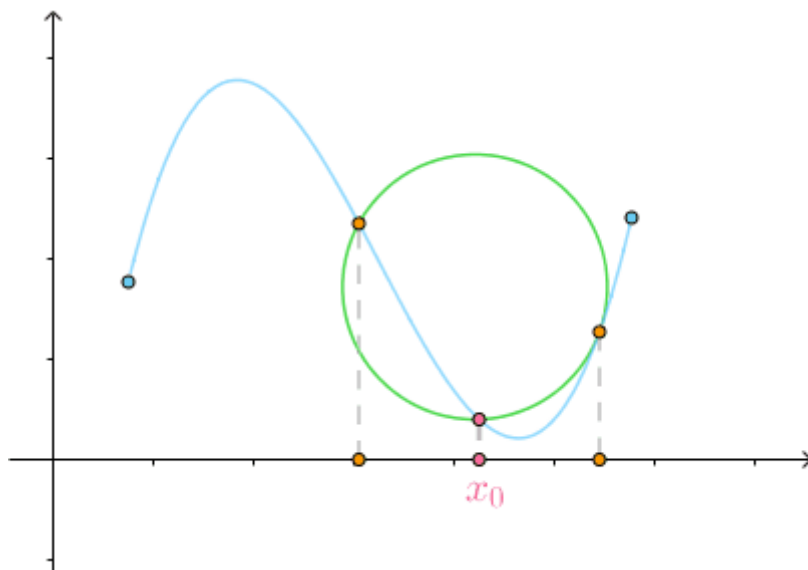


怎么来表示某一条曲线的弯曲程度呢？

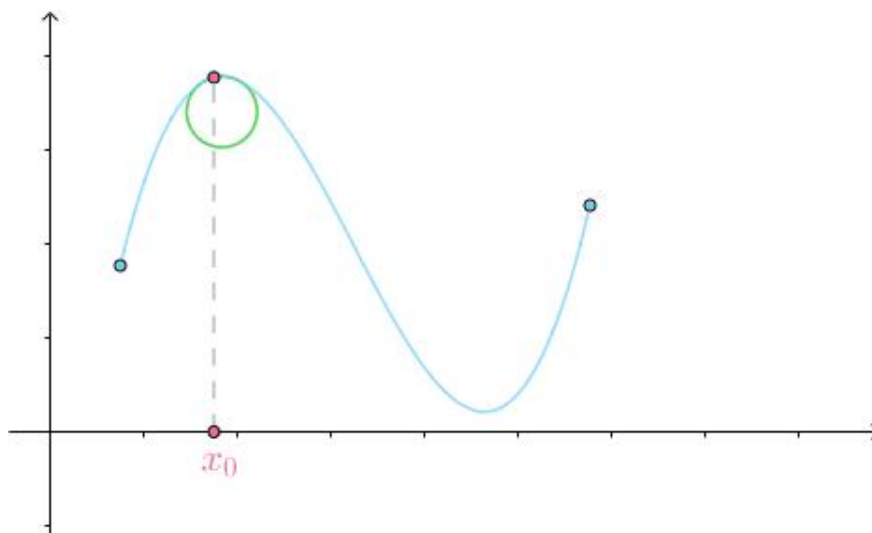
我们知道三点确定一个圆：



当 δ 趋近于0时，我们可以得到曲线在 x_0 处的密切圆，也就是曲线在该点的圆近似：



另外我们也可以观察到，在曲线比较平坦的位置，密切圆较大，在曲线比较弯曲的地方，密切圆较小，



因此，我们通过密切圆的曲率来定义曲线的曲率，定义如下：

已知函数 $f(x)$ 在 x_0 点有二阶导数 $f''(x_0)$ ，且 $f''(x_0) \neq 0$ ，则此点有密切圆，其半径为：

$$r = \frac{(1 + (f'(x_0))^2)^{\frac{3}{2}}}{|f''(x_0)|}$$

此时，曲线的曲率也就是密切圆的曲率，为：

$$K = \frac{1}{r} = \frac{|f''(x_0)|}{(1 + (f'(x_0))^2)^{\frac{3}{2}}}$$

所以密切圆也称为曲线的曲率圆，半径 r 称为曲率半径。

2.实现

我们根据上述的计算曲率半径的方法，代码实现如下：

```
def cal_radius(img, left_fit, right_fit):

    # 图像中像素个数与实际中距离的比率
    # 沿车行进的方向长度大概覆盖了30米，按照美国高速公路的标准，宽度为3.7米（经验值）
    ym_per_pix = 30 / 720 # y方向像素个数与距离的比例
    xm_per_pix = 3.7 / 700 # x方向像素个数与距离的比例

    # 计算得到曲线上的每个点
    left_y_axis = np.linspace(0, img.shape[0], img.shape[0] - 1)
    left_x_axis = left_fit[0] * left_y_axis ** 2 + left_fit[1] * left_y_axis + left_fit[2]
    right_y_axis = np.linspace(0, img.shape[0], img.shape[0] - 1)
    right_x_axis = right_fit[0] * right_y_axis ** 2 + right_fit[1] * right_y_axis + right_fit[2]

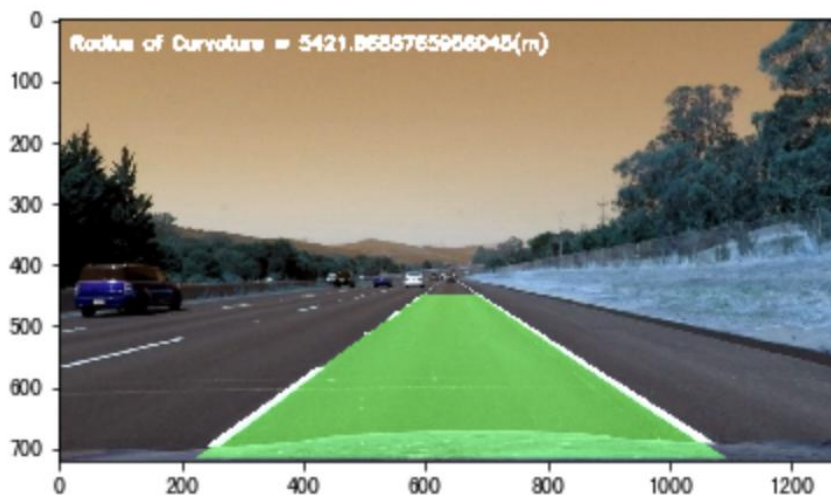
    # 获取真实环境中的曲线
    left_fit_cr = np.polyfit(left_y_axis * ym_per_pix, left_x_axis * xm_per_pix, 2)
    right_fit_cr = np.polyfit(right_y_axis * ym_per_pix, right_x_axis * xm_per_pix, 2)

    # 获得真实环境中的曲线曲率
    left_curverad = ((1 + (2 * left_fit_cr[0] * left_y_axis * ym_per_pix + left_fit_cr[1] ** 2) * ym_per_pix ** 2) / (2 * left_fit_cr[0]))
    right_curverad = ((1 + (2 * right_fit_cr[0] * right_y_axis * ym_per_pix + right_fit_cr[1] ** 2) * ym_per_pix ** 2) / (2 * right_fit_cr[0]))

    # 在图像上显示曲率
    cv2.putText(img, 'Radius of Curvature = {:.1f}(m)'.format(np.mean([left_curverad, right_curverad])),
                (255, 255, 255), 5)

    return img
```

显示效果：



计算偏离中心的距离：

```

# 1. 定义函数计算图像的中心点位置
def cal_line__center(img):
    undistort_img = img_undistort(img, mtx, dist)
    rigin_pipeline_img = pipeline(undistort_img)
    transform_img = img_perspect_transform(rigin_pipeline_img, M)
    left_fit, right_fit = cal_line_param(transform_img)
    y_max = img.shape[0]
    left_x = left_fit[0] * y_max ** 2 + left_fit[1] * y_max + left_fit[2]
    right_x = right_fit[0] * y_max ** 2 + right_fit[1] * y_max + right_fit[2]
    return (left_x + right_x) / 2

# 2. 假设straight_lines2_line.jpg, 这张图片是位于车道的中央, 实际情况可以根据测量验证。
img = cv2.imread("./test/straight_lines2_line.jpg")
lane_center = cal_line__center(img)
print("车道的中心点为: {}".format(lane_center))

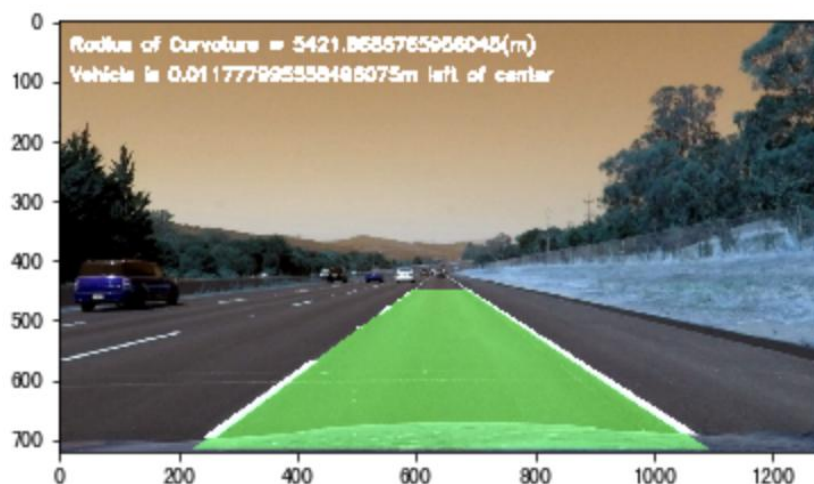
# 3. 计算偏离中心的距离
def cal_center_departure(img, left_fit, right_fit):

    # 计算中心点
    y_max = img.shape[0]
    left_x = left_fit[0] * y_max ** 2 + left_fit[1] * y_max + left_fit[2]
    right_x = right_fit[0] * y_max ** 2 + right_fit[1] * y_max + right_fit[2]
    xm_per_pix = 3.7 / 700
    center_depart = ((left_x + right_x) / 2 - lane_center) * xm_per_pix

    # 在图像上显示偏移
    if center_depart > 0:
        cv2.putText(img, 'Vehicle is {}m right of center'.format(center_depart),
                    (255, 255, 255), 5)
    elif center_depart < 0:
        cv2.putText(img, 'Vehicle is {}m left of center'.format(-center_depart),
                    (255, 255, 255), 5)
    else:
        cv2.putText(img, 'Vehicle is in the center', (20, 100), cv2.FONT_ITALIC, 1)
    return img

```

显示效果如下:



总结

1. 曲率是表示曲线的弯曲程度, 在这里是计算车道的弯曲程度

2.1. numba

2. 偏离中心的距离：利用已知的在中心的图像计算其他图像的偏离距离

5.6. 在视频中检测车道线

学习目标

- 了解在视频中检测车道线的方法

在前面几节中一步步完成摄像机标定、图像畸变校正、透视变换、提取车道线、检测车道线、计算曲率和偏离距离后，在图像上实现了复杂环境下的车道线检测算法。现在我们将视频转化为图片，然后一帧帧地对视频数据进行处理，然后将车道线检测结果存为另一段视频，代码如下：

首先将前面的方法进行汇总：

```
def process_image(img):
    # 1. 图像去畸变
    undistort_img = img_undistort(img, mtx, dist)
    # 2. 车道线检测
    rigin_pipeline_img = pipeline(undistort_img)
    # 3. 透视变换
    transform_img = img_perspect_transform(rigin_pipeline_img, M)
    # 4. 精确定位车道线，并拟合
    left_fit, right_fit = cal_line_param(transform_img)
    # 5. 绘制车道区域
    result = fill_lane_poly(transform_img, left_fit, right_fit)
    # 6. 反投影
    transform_img_inverse = img_perspect_transform(result, M_inverse)
    # 7. 计算曲率半径和偏离中心的距离
    transform_img_inverse = cal_radius(transform_img_inverse, left_fit, right_fit)
    transform_img_inverse = cal_center_departure(transform_img_inverse, left_fit, right_fit)
    # 8. 将检测结果与原始图像叠加
    transform_img_inverse = cv2.addWeighted(undistort_img, 1, transform_img_inverse, 0.5, 0)
    return transform_img_inverse
```

接下来，读取视频并调用上述方法：

```
clip1 = VideoFileClip("project_video.mp4")
white_clip = clip1.fl_image(process_image)
white_clip.write_videofile("output.mp4", audio=False)
```

处理结果如下所示：